AUTUMN **MID SEMESTER EXAMINATION-2025**

School of Computer Engineering
Kalinga Institute of Industrial Technology, deemed to be University

Software Engineering
CS31001
Solution Scheme

1. Answers

A. Just because a program compile doesn't mean it will work. A syntactically correct program can still do the wrong thing if the logic is flawed, leading to unexpected outputs. In the wider context *of a software product, unclear or shifting requirements can also doom the result.* When *project requirements lack clarity or change frequently, teams can't align their work with the desired outcomes and end up building features that don't meet user needs*. So, a *product may "work" from a compiler's point of view but still fail because of wrong logic or because it doesn't solve the right problem*.

B. Traceability in an SRS means being able to follow each requirement from its origin through design, code and testing. Requirements should be linked to other artifacts such as use cases and user stories to make sure they are all addressed. *A well-structured SRS boosts traceability by linking requirements to design, code and tests, and traceable, version-controlled requirements help meet regulatory standards.* This linking makes it easier to verify that every requirement has been implemented and tested and to assess the impact of changes, which is why traceability is so important.

C. Control-flow-based design focuses on the sequence of operations rather than on data relationships. For a *small module this can be helpful, but control-flow graphs become complex for big programs, are hard to understand, and provide no information about data they just show program flow*. Such diagrams are *also not scalable and become messy for large projects.* Using only control-flow-based design for a large system can therefore *lead to tangled diagrams that are hard to maintain and that omit important information* about data and interactions.

D. Agile approaches emphasize flexibility, frequent feedback and minimal upfront documentation. For safety-critical systems such as medical devices, some researchers have argued that this may not be appropriate. Shortly after agile methods became popular, the software engineering community suggested *they were not suited for safety-critical systems because of a perceived lack of documentation*, formal specification and detailed planning. Other comparisons of agile and plan-driven methods concluded that life-critical systems need stable requirements and that an agile approach may not be suitable. This is why many organizations use hybrid models or add extra controls when applying agile techniques to safety-critical work.

E. A context diagram is the highest-level view of a system used in structured analysis. It shows the system as a single process and depicts its interactions with external entities such as users, other systems or processes. Context diagrams serve as a foundational tool for designers and stakeholders, helping them grasp the scope and boundaries of the system. These diagrams define the system's boundaries by highlighting interactions with external entities and aid in identifying functional and non-functional requirements. In other words, a context diagram sets the stage for more detailed analysis by clarifying what is inside the system, what is outside, and how data flows between them, which is essential for communication and requirement gathering.

LONG QUESTIONS and ANSWERS

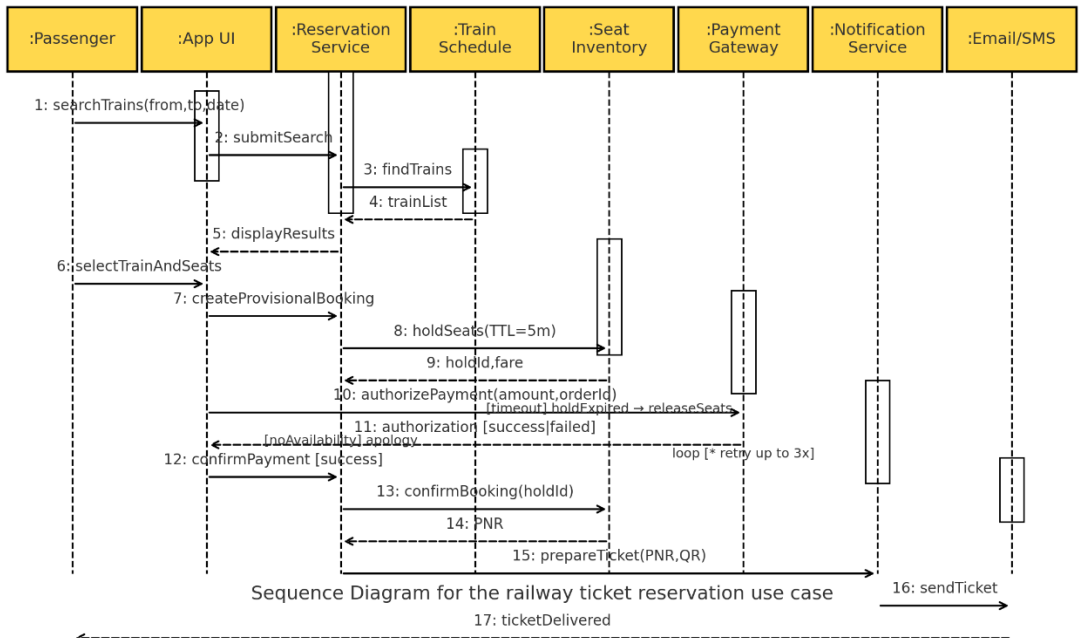
2. Redesigning a legacy system to improve its maintainability and scalability involves choosing a design approach that supports clear organization and future growth. Function-oriented design divides the program into a series of smaller functions, linked together by the flow of data. This top-down method often relies on shared global data and lets functions change system state as they run. While it can work well for straightforward computational tasks, it tends to tangle modules together. A change in one function can have unintended effects on others because they all rely on the same shared state. Over time, code related to one concept gets scattered across many different parts of the system. This scattering creates strong dependencies between modules and weakens the focus within each module. Such designs are hard to test, difficult to modify, and they don't scale gracefully.

In contrast, ***object-oriented design organizes code around the data and real-world entities the system deals with****. It encapsulates data and behavior into objects, with classes defining these objects and relationships among them. Each object hides its internal state and exposes operations through well-defined methods. Because data and methods are bundled together, modules interact through clear interfaces instead of directly manipulating shared data. This keeps modules more independent and improves cohesion: all the responsibilities related to a given concept live together in one place. It reduces coupling too, because changes inside an object usually don't affect other modules. Inheritance and polymorphism allow new functionality to be added by deriving new classes or overriding methods, rather than by rewriting existing code. That makes extending the system easier and less risky.*

High cohesion and low coupling are widely recognized as hallmarks of well-structured software. An object-oriented design achieves these qualities by keeping each class focused on a single purpose and by limiting interactions to clearly defined interfaces. As a result, the system becomes easier to understand, modify and scale. Function-oriented designs, with their emphasis on shared data and procedural decomposition, struggle to achieve the same clarity and independence. For a legacy system that needs to evolve, object-oriented

design offers the better foundation for maintainability and scalability while still allowing the use of simple functions internally for algorithmic tasks.

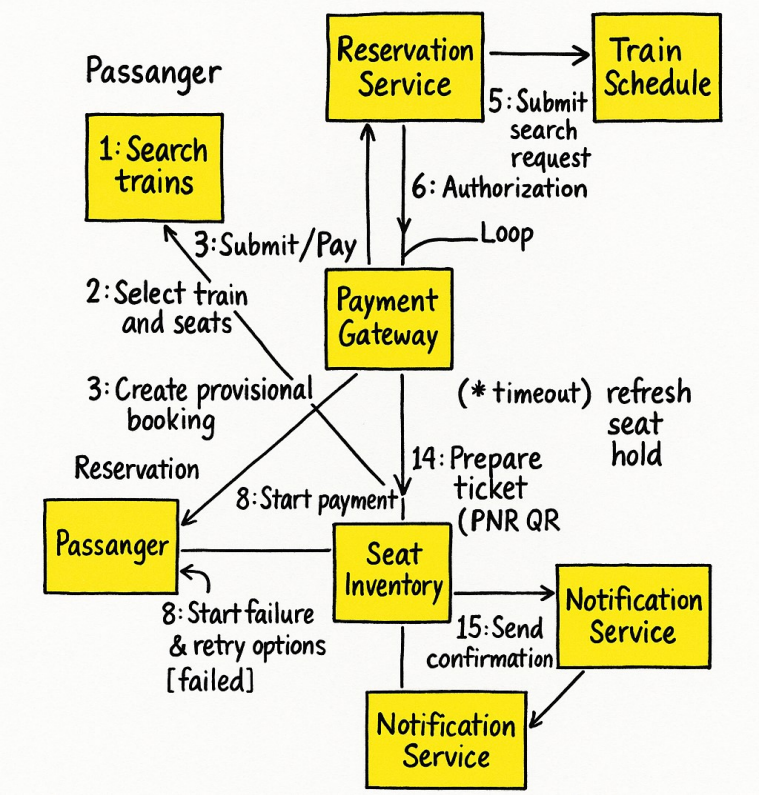
3. **Sequence diagram:** The sequence diagram illustrates the *step-by-step interaction* between the *Passenger, App UI, Reservation Service, Train Schedule, Seat Inventory, Payment Gateway, Notification Service, and Email/SMS* components during a railway ticket booking process. The process begins when the passenger searches for trains by entering journey details into the App UI. The App UI sends this request to the Reservation Service, which fetches train availability from the Train Schedule system and returns the results to the user. After selecting the desired train and seats, the App UI initiates a provisional booking request through the Reservation Service, which places a temporary hold on the seats via the Seat Inventory service and receives a hold ID and fare details. The passenger then proceeds with payment, where the App UI interacts with the Payment Gateway for authorization, allowing up to three retry attempts in case of failure. On successful payment, the Reservation Service confirms the booking with the Seat Inventory, generates a PNR, and forwards the details to the Notification Service, which sends the ticket via Email/SMS to the passenger. If the payment fails or times out, the held seats are released automatically. This diagram clearly demonstrates the flow of messages, conditional paths, retries, and the interaction among different services required to complete a railway ticket reservation.



Sequence Diagram for the railway ticket reservation use case

Collaboration (communication) diagram: The collaboration diagram shows the same railway ticket booking process as the sequence diagram but focuses on the relationships and interactions between system components rather than the chronological order of events. The passenger starts by requesting train availability through the App UI, which

communicates with the Reservation Service. The Reservation Service interacts with the Train Schedule system to retrieve train information and sends the available options back to the App UI. Once the passenger selects seats, the Reservation Service coordinates with the Seat Inventory to temporarily hold them while awaiting payment confirmation. The Payment Gateway processes the passenger's payment, allowing retry options in case of failure. On successful payment, the Reservation Service confirms the booking with the Seat Inventory, creates the PNR, and sends the information to the Notification Service, which finally delivers the ticket to the passenger through Email/SMS. Additionally, the diagram highlights alternative flows such as failed payment handling, automatic seat release on timeout, and retry mechanisms. This representation emphasizes how different components communicate with one another and provides a clear overview of the booking process at a system-to-system interaction level.



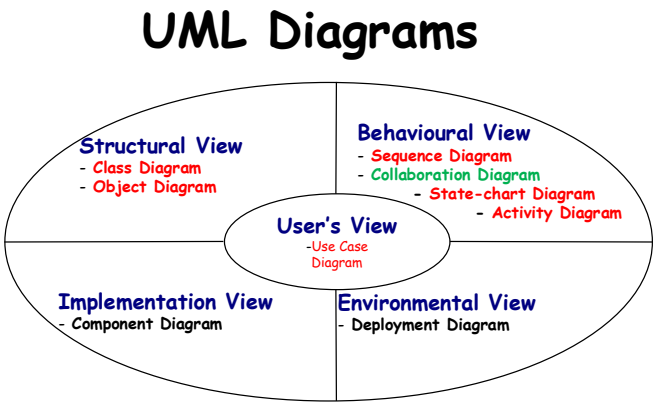
4. The classical waterfall model is a *poor fit for fast-changing web applications because it locks decisions early*, relies on complete upfront requirements, and pushes testing and real user feedback to the end, which makes change slow and expensive. In practice this leads to missed or outdated features by the time they ship, *late discovery of integration and quality issues, and long delays between what stakeholders* ask for and what they actually see working. Teams also *lose agility to respond to market shifts, since revisiting earlier phases triggers cascading rework across requirements, design, and code*.

A ***more suitable choice is an Agile, iterative approach like Scrum or Kanban***. These methods plan in small chunks, deliver working software frequently, and adjust priorities based on *ongoing feedback, which helps catch misunderstandings early, reduce rework, and keep the product aligned with real user needs. They naturally support continuous integration, automated testing, and continuous delivery, so quality and deployment readiness improve with every iteration. For areas with high uncertainty like performance, compliance, or novel tech add short, time-boxed risk spikes or prototypes within iterations to de-risk early before committing to full builds.* In short, use Agile iterations to deliver value quickly and learn continuously, and layer in targeted risk-reduction work where needed; this combination matches the pace and uncertainty of modern web development far better than the sequential, change-averse nature of waterfall.

5. Short notes

a. **Verification and Validation:** Verification checks whether the software is being built correctly according to specifications and design think reviews, walkthroughs, static analysis, and unit tests that ensure each artifact (requirements, design, code) is consistent and complete. Validation checks whether the right product is being built for real user needs think executing the system through system tests, acceptance tests, usability tests, and field trials to confirm it solves the intended problem in the intended context. In short: verification asks "are we building it right?" while validation asks "are we building the right thing?", and both together reduce defects early, prevent costly rework, and increase user satisfaction.

b. **UML:** The *Unified Modeling Language* is a standardized visual language to describe, design, and document software systems, helping teams communicate structure and behavior before and during implementation.



Diagrams and views in UML

54

It offers structural diagrams (like class, component, and deployment diagrams) to model static architecture, and behavioral diagrams (like use case, sequence, activity,

and state machine diagrams) to capture interactions, workflows, and lifecycles. Used pragmatically, UML clarifies requirements, aligns stakeholders, de-risks complex designs, and supports traceability from high-level use cases to detailed design and testing.

c. **Scrum:** Scrum is an Agile framework for delivering value iteratively through short, fixed-length sprints (often 1-2 weeks) with a clear, prioritized backlog. Roles include a Product Owner who defines and orders value, a Scrum Master who enables the process and removes impediments, and a cross-functional Development Team that plans, builds, and tests a potentially shippable increment each sprint. Key events, Sprint Planning, Daily Scrum, Sprint Review, and Sprint Retrospective, create a rhythm of transparency, inspection, and adaptation, enabling continuous feedback, predictable delivery, and steady improvements in both product and process.